

Белорусский Государственный Университет
Факультет прикладной математики и информатики

Компьютерная графика

Кулинкович Виктория Александровна



Основы обработки цифровых изображений.

- Гистограмма изображения
- Сегментация изображений
- Пороговая обработка
- Бинаризация
- Морфологическая обработка
- Библиотека OpenCV

Гистограмма изображения

Гистограмма показывает распределение на изображении пикселей с различными уровнями яркости из заданного диапазона, т.е. представляет собой дискретную функцию, несущую информацию о числе элементов изображения, имеющих заданный уровень яркости.

$$h(j_i) = h_i, \quad i = 0, 1, \ldots, L - 1$$

h_i – число элементов изображения, имеющих яркость j_i

Гистограмма полутонового изображения

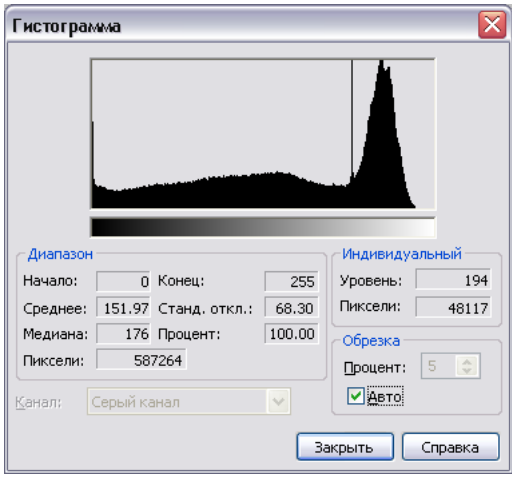
$$h(i) = h_i, \quad i = 0, 1, \ldots, 255 \qquad \textbf{(1)}$$

$$h_i = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} g_i(m,n), \qquad i = 0, 1, \ldots, 255 \qquad \textbf{(2)}$$

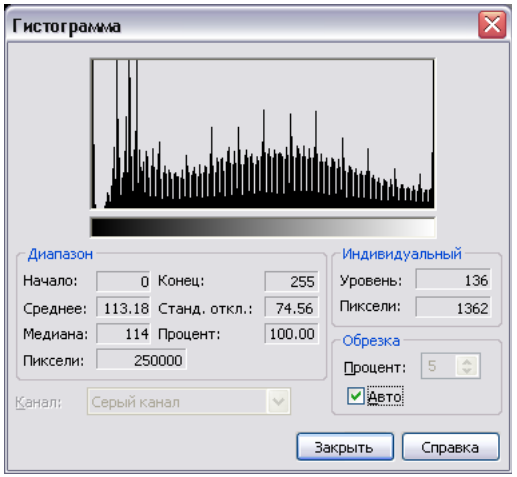
$$g_i(m, n) = \begin{cases} 1, & \text{если } f(m, n) = i, \\ 0 & \text{в противном случае.} \end{cases} \qquad \textbf{(3)}$$

Гистограмма изображения



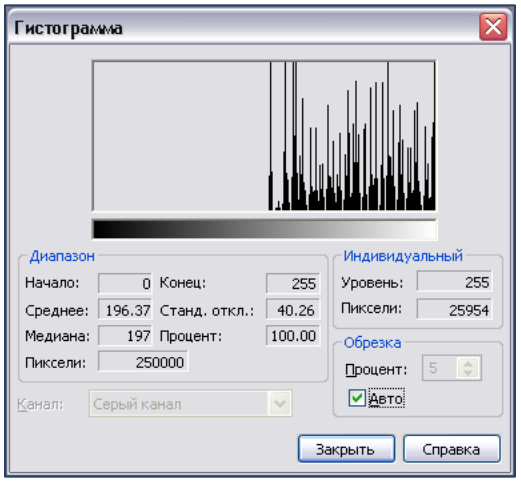


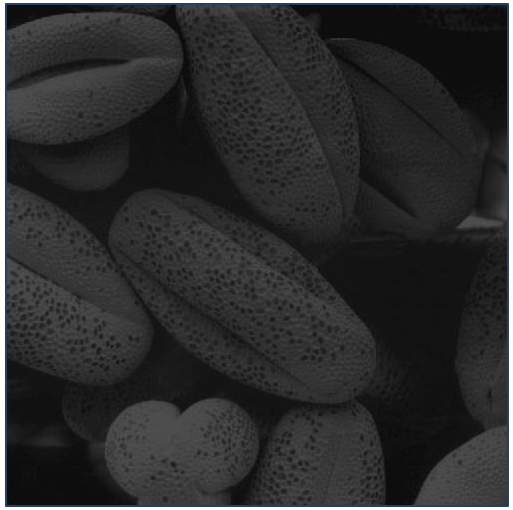


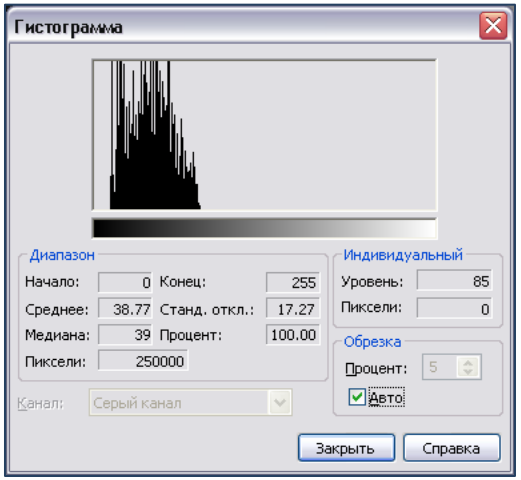


Гистограмма изображения



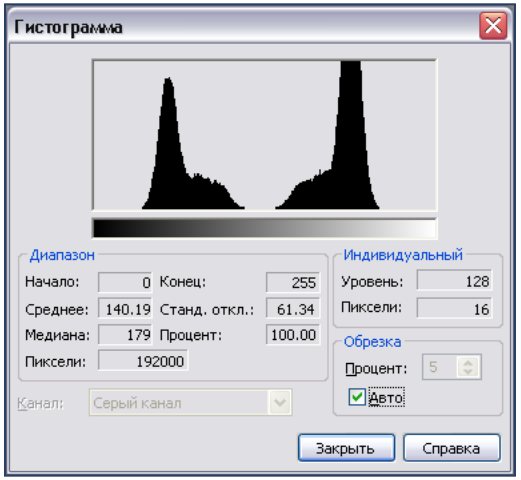




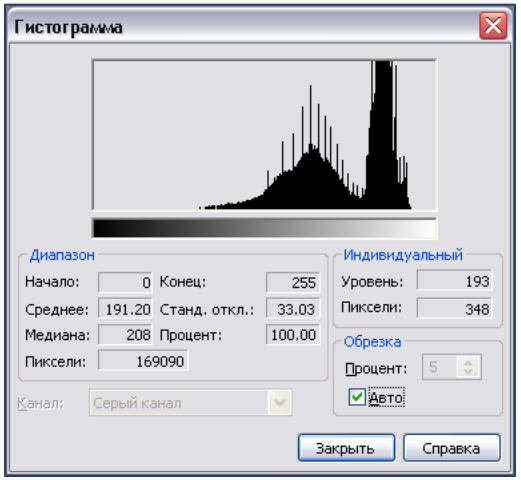


Гистограмма изображения









Нормализация гистограммы изображения

$$h'(i) = \frac{h_i}{H}$$

где h_i – количество пикселей с яркостью i

$$H = \sum_{i=0}^{255} h_i$$ – общее число пикселей на изображении

$h'(i)$ – вероятность появления пикселя с яркостью i

$$0 \le h'(i) \le 1$$

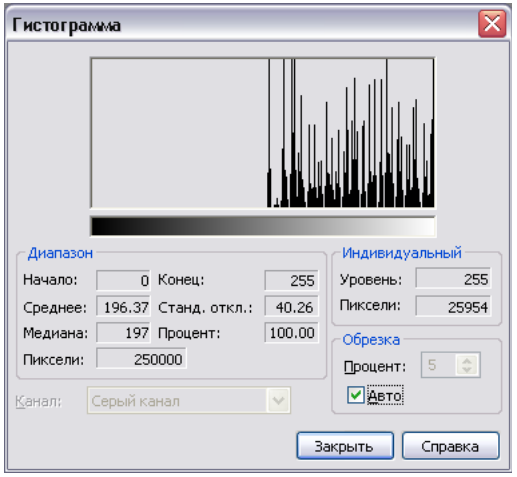
$$\sum_i h'(i) = 1$$

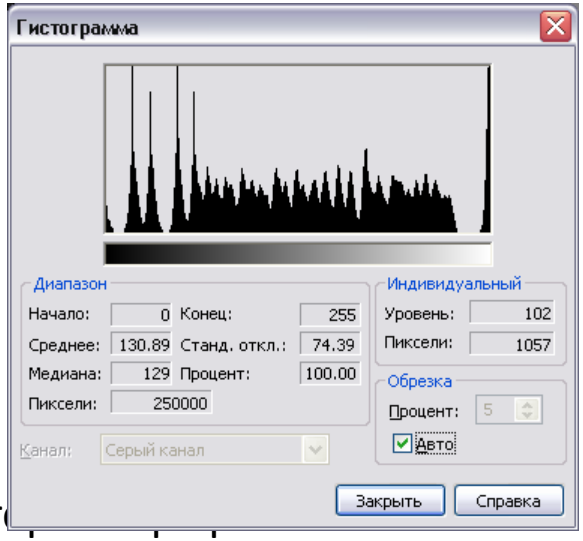
Эквализация гистограммы изображения

Процесс ***эквализации*** (***выравнивания***) гистограммы предполагает, что идеальная гистограмма должна иметь форму, близкую к прямоугольнику, т.е. ни один уровень яркости не должен преобладать над другими. Данный метод формирует на выходе изображение, уровни яркости которого равномерно покрывают весь диапазон возможных значений, и является полностью «автоматическим».

Эквализация гистограммы



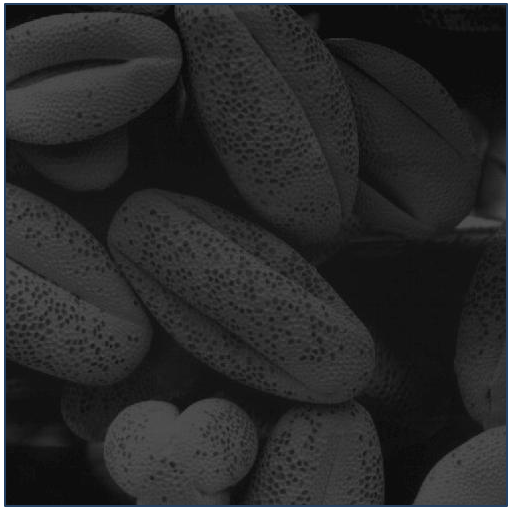


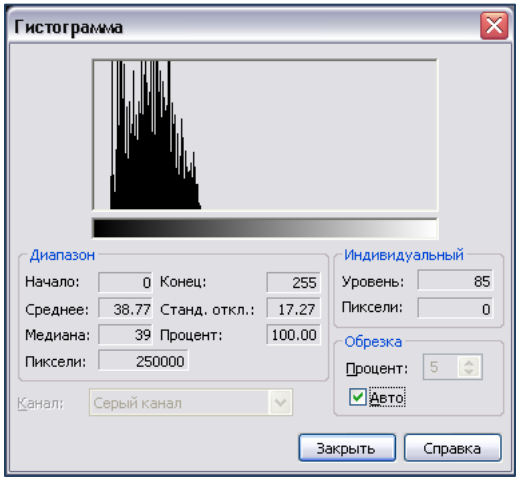


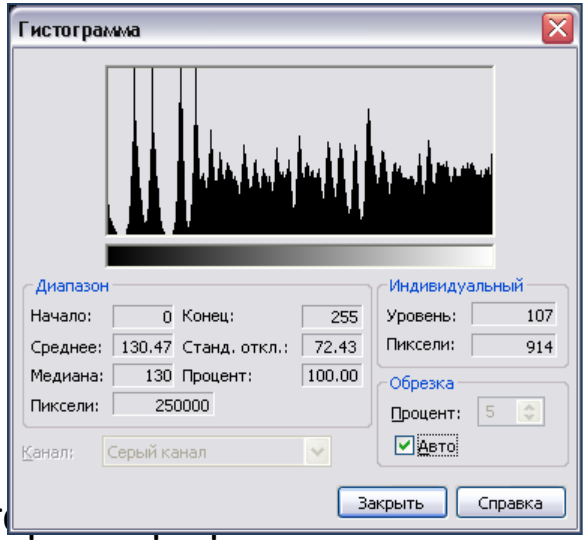


Компьют

Эквализация гистограммы



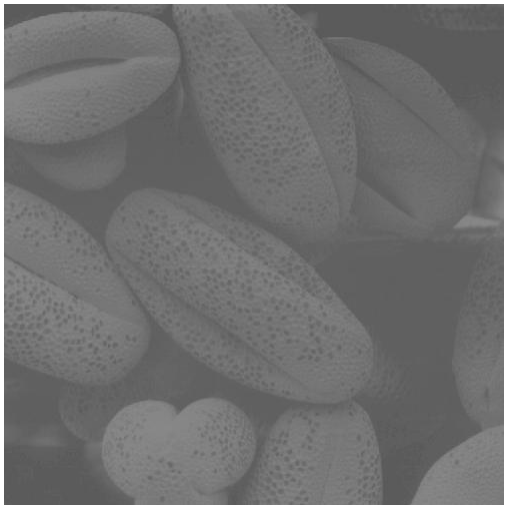


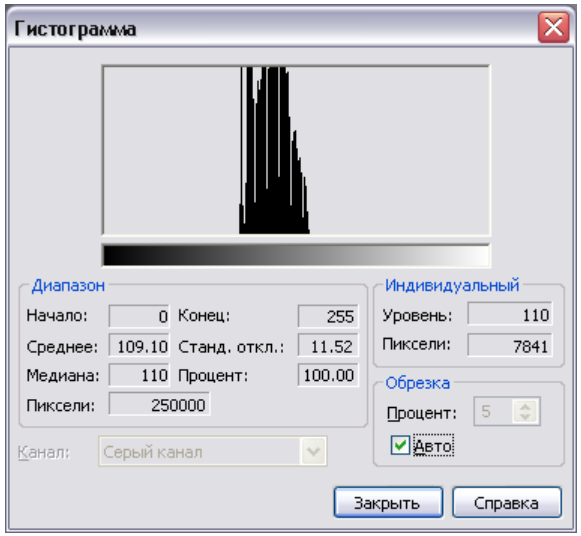


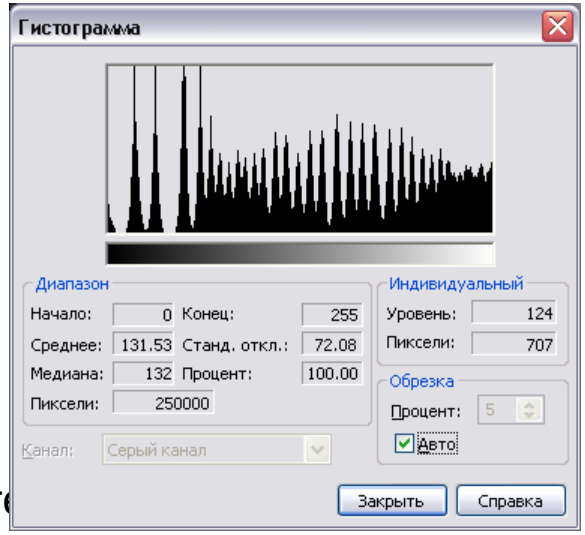


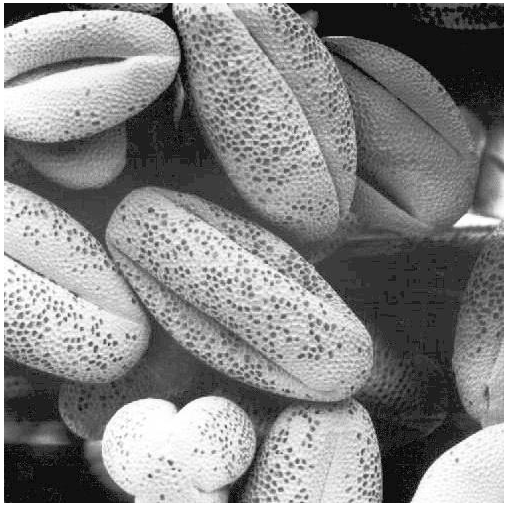
Компьют

Эквализация гистограммы









Компьюте

Эквализация (линейное выравнивание) гистограммы изображения

f(m,n) – исходное полутоновое изображение размера M×N

1. Построение гистограммы исходного изображения

$$h(i) = h_i,\ i = 0,1,\ldots,255$$

2. Нормализация гистограммы

$$H = \sum_{i=0}^{255} h_i \qquad h'(i) = \frac{h_i}{H},\ i = 0,1,\ldots,255$$

3. Построение кумулятивной гистограммы

$$Sh(i) = \sum_{j=1}^{i} h'(j),\quad i = 0,1,\ldots,255$$

4. Вычисление новых значений яркости каждого пикселя изображения

$$f'(m,n) = 255 \cdot Sh(f(m,n))$$

Эквализация (нелинейное выравнивание) гистограммы изображения

$H = M \times N$ – площадь изображения (т.е. количество пикселей)
L – число допустимых уровней яркости (256 в нашем случае).

Необходимо, чтобы на каждый уровень яркости приходилось по H / L пикселей.
Если значение яркости некоторого уровня i в k раз больше среднего H / L, то этот уровень следует отобразить в k отдельных уровней.
Если значение яркости некоторого уровня i в k раз меньше среднего H / L, то этот уровень следует объединить с k соседними уровнями с примерно равным значением.

Эквализация (выравнивание) гистограммы изображения

Процедура преобразования гистограммы может применяться как к изображению в целом, так и к отдельным его фрагментам. Последнее может быть полезным при обработке нестационарных изображений, содержание которых существенно различается по своим характеристикам на различных участках. В этом случае лучшего эффекта можно добиться, применяя гистограммную обработку к отдельным участкам.

Методы повышения контраста

- Линейное контрастирование

$$f'(m,n) = \frac{255}{f_{\max} - f_{\min}}(f(m,n) - f_{\min})$$

- Эквализация гистограммы

Задание для л/р: реализовать оба метода повышения контраста для полутоновых изображений. Сравнить результат.

Эквализация гистограммы цветного изображения

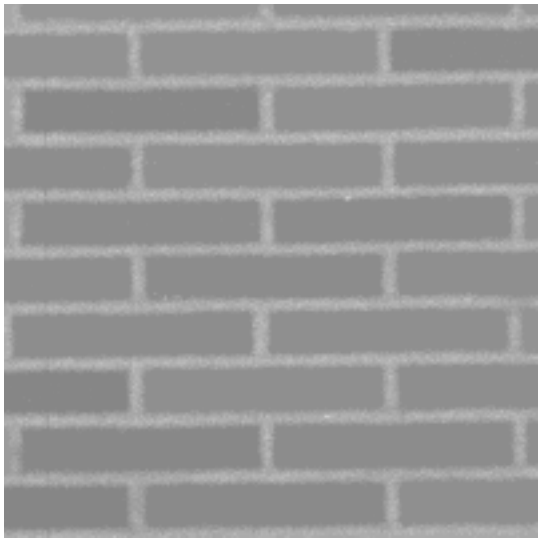
- Работа с цветом в пространстве RGB. Процедура эквализации гистограммы для каждой цветовой компоненты в отдельности.

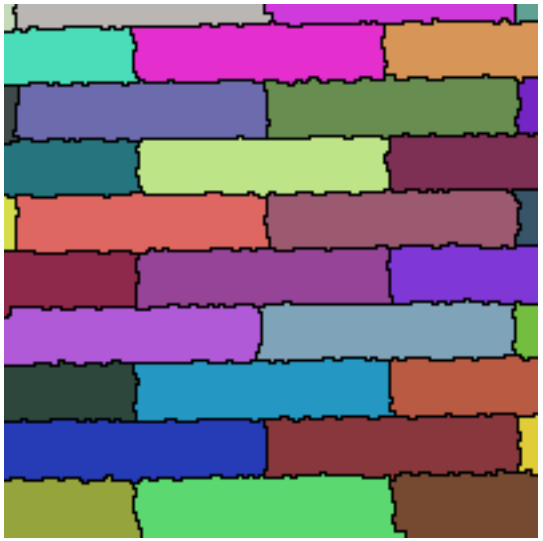
- Переход к цветовому пространству HSV. Применение процедуры эквализации только для компоненты яркости (V).

Задание для л/р: реализовать оба варианта эквализации гистограммы цветного изображения. Сравнить результат.

Сегментация изображений

Сегментация изображений – процесс разделения изображения на однородные в определенном смысле области.





Пороговая обработка

Пороговые преобразования занимают центральное место в прикладных задачах сегментации изображений.

Пороговая обработка – самый простой метод сегментации, ориентированный на обработку изображений, отдельные однородные участки которых различаются средней яркостью.

Пороговая обработка
Бинаризация

Бинаризация - операция порогового разделения, которая в результате дает бинарное изображение. Целью операции бинаризации является уменьшение количества информации, содержащейся на изображении. В процессе бинаризации исходное полутоновое изображение, имеющее L уровней яркости, преобразуется в черно-белое изображение, пиксели которого имеют только два значения – 0 и 1. Поскольку количество информации в бинарном изображении почти на порядок меньше, чем в совпадающим с ним по размерам полутоновом изображении, то бинарное изображение легче обрабатывать, хранить и пересылать.

Пороговая обработка
Бинаризация

f(x,y) – изображение (f(m,n))
T – порог
f(x,y)>T - точка объекта
f(x,y)<T - точка фона

В общем случае:
T – T (x,y,p(x,y),f)

• глобальная – значение T одинаково для всех точек изображения, т.е. T=T(f)
•локальная - значение T зависит от пространственных координат x и y, т.е. T=T(x,y,f)
•адаптивная - значение T зависит от локальных характеристик окрестности рассматриваемой точки, т.е. T=T(x,y,f, p(x,y))



Бинаризация

- Бинаризация с нижним порогом
- Бинаризация с верхним порогом
- Бинаризация с двойным ограничением
- Бинаризация по условию*
- Неполная пороговая обработка*
- Многоуровневое пороговое преобразование*

* - Данная операция формирует изображение, не являющееся бинарным, но состоящее из сегментов с различной яркостью

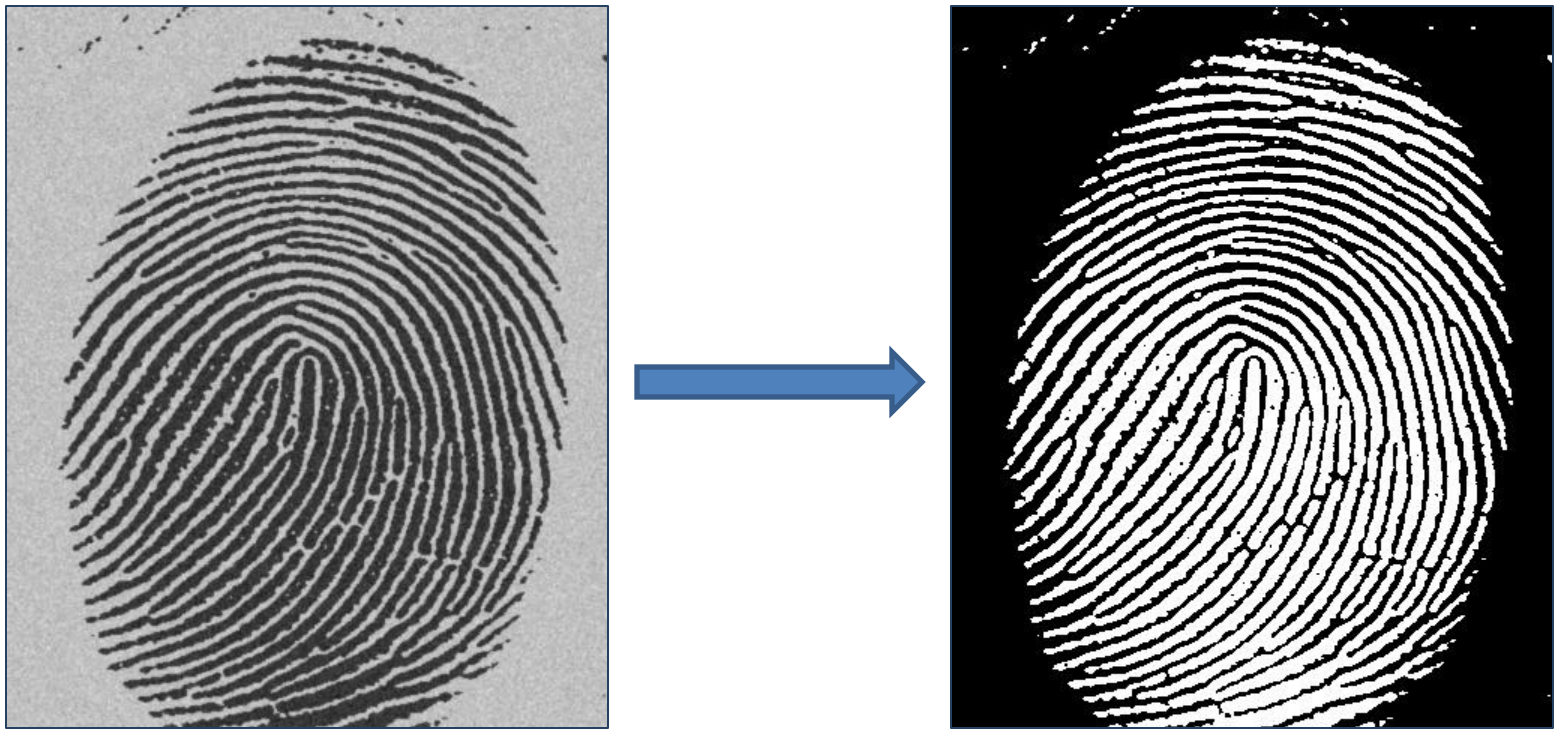
Бинаризация с нижним порогом

$$f'(m,n) = \begin{cases} 0, & f(m,n) \le t; \\ 1, & f(m,n) > t, \end{cases}$$



Бинаризация с верхним порогом

$$f'(m, n) = \begin{cases} 0, & f(m, n) \geq t; \\ 1, & f(m, n) < t. \end{cases}$$



Бинаризация с двойным ограничением

$$f'(m, n) = \begin{cases} 0, & f(m, n) \le t_1; \\ 1, & t_1 < f(m, n) \le t_2; \quad t_1 < t_2 \\ 0, & f(m, n) > t_2, \end{cases}$$

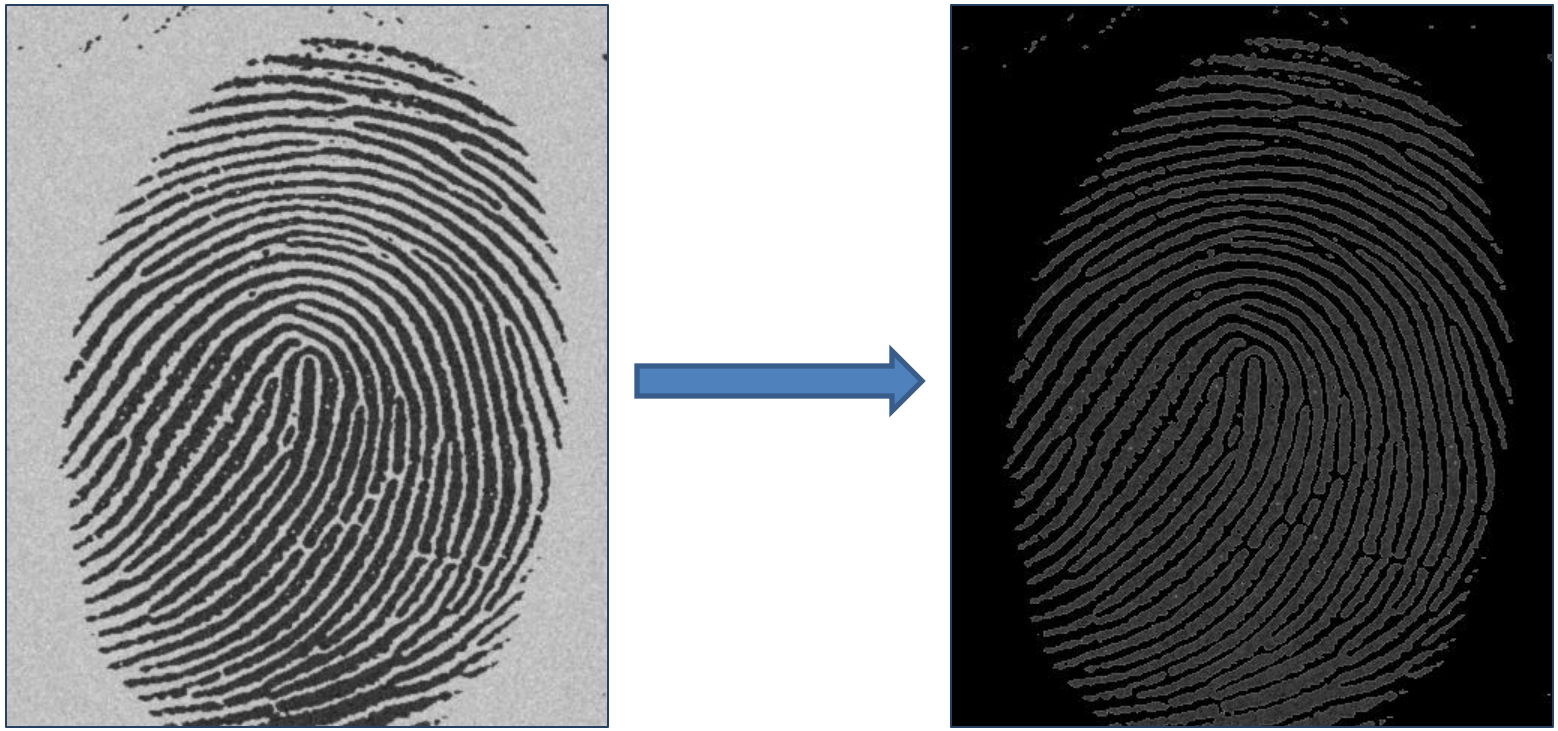
Бинаризация по условию

$$f'(m, n) = \begin{cases} 0, & f(m, n) \le t_1; \\ s, & t_1 < f(m, n) \le t_2; \\ 0, & f(m, n) > t_2, \end{cases}$$

Неполная пороговая обработка

$$f'(m,n) = \begin{cases} f(m,n), & f(m,n) > t; \\ 0, & f(m,n) \le t. \end{cases}$$

$$f'(m,n) = \begin{cases} f(m,n), & f(m,n) < t; \\ 255, & f(m,n) \ge t. \end{cases}$$

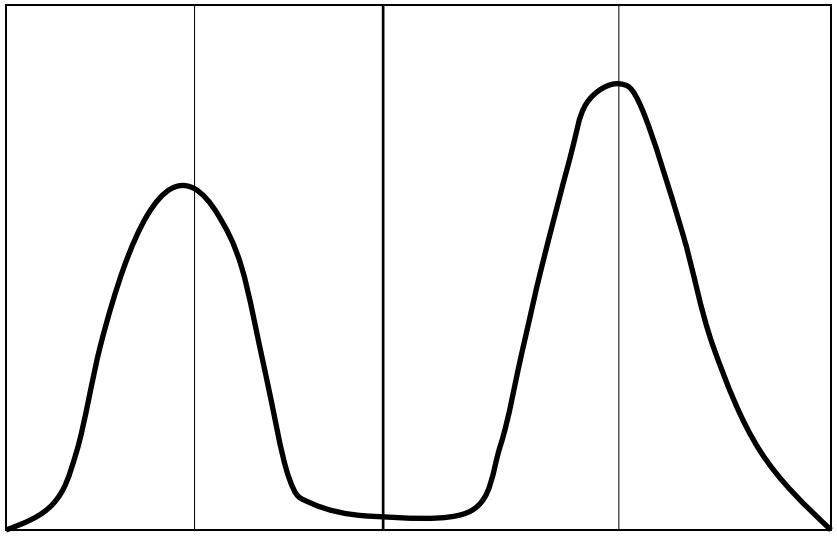


Глобальная пороговая обработка. Выбор порога.

- *Использование гистограммы изображения*
- *Определение порога на основе градиента яркости изображения*
- *Метод Отсу (определение оптимального порога)*
- *Метод использования энтропии гистограммы*

Глобальная пороговая обработка. Выбор порога с использованием гистограммы.

Использование гистограммы.



T

Глобальная пороговая обработка. Выбор порога с использованием гистограммы.

Использование гистограммы.

1 Выбирается некоторая начальная оценка значения порога ***t***.

2 Выполняется сегментация изображения с помощью порога ***t*** . В результате образуются две группы пикселей: G_1, состоящая из пикселей с яркостью больше ***t***, и G_2, состоящая из пикселей с яркостью меньше или равной ***t*** .

3 Вычисляются средние значения μ_1 и μ_2 яркостей пикселей по областям G_1 и G_2 соответственно.

4 Вычисляется новое значение порога

.

$$t = \frac{\mu_1 + \mu_2}{2}$$

5 Повторяются шаги со 2-го по 4-й до тех пор, пока разница значений порога t в соседних итерациях не окажется наперед меньше заданного параметра ε .

Глобальная пороговая обработка. Выбор порога на основе градиента.

$$\nabla \mathbf{f} = \begin{bmatrix} G_x \\ G_y \end{bmatrix} = \begin{bmatrix} \dfrac{\partial f}{\partial x} \\ \dfrac{\partial f}{\partial y} \end{bmatrix}$$

$$G_x = \frac{\partial f}{\partial x} = f(x+1, y) - f(x, y);$$

$$G_y = \frac{\partial f}{\partial y} = f(x, y+1) - f(x, y).$$

$$G(x, y) = \max\{|G_x|, |G_y|\}$$

$$t = \frac{\sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y)\, G(x, y)}{\sum_{x=0}^{M-1} \sum_{y=0}^{N-1} G(x, y)}$$

Глобальная пороговая обработка. Метод Отсу

С помощью данного метода вычисляется порог T, который минимизирует внутриклассовую дисперсию (или максимизирует межклассовую дисперсию) Т.е., вычисленный порог минимизирует среднюю ошибку сегментации, т.е. среднюю ошибку от принятия решения о принадлежности пикселей изображения объекту или фону.

Глобальная пороговая обработка. Метод Отсу

1. Построение нормализованной гистограммы изображения

$$h(i) = h_i,\ i = 0,1,\ldots,255,\quad \sum_{i=0}^{255} h_i = 1$$

2. Разбиение на два класса пикселей G_1 и G_2 с помощью некоторого значения порога T. Вычисление межклассовой дисперсии:

$$S(T) = P_1(T) \cdot P_2(T) \cdot (M_1(T) - M_2(T))^2$$

где P_1 и P_2 – вероятности двух классов,
M_1 и M_2 – средние значения яркости

$$P_1(T) = \sum_{i=0}^{T} h(i),\ \ P_2(T) = 1 - P_1(T)$$

$$M_1(T) = \sum_{i=0}^{T} i \cdot h(i) / P_1(T),\ \ M_2(T) = \sum_{i=T+1}^{255} i \cdot h(i) / P_2(T)$$

3. Максимизация *S(T):* $T_{opt} = \arg\left\{ \max_{0 \le T \le 255} (S(T)) \right\}$

Глобальная пороговая обработка. Метод Отсу

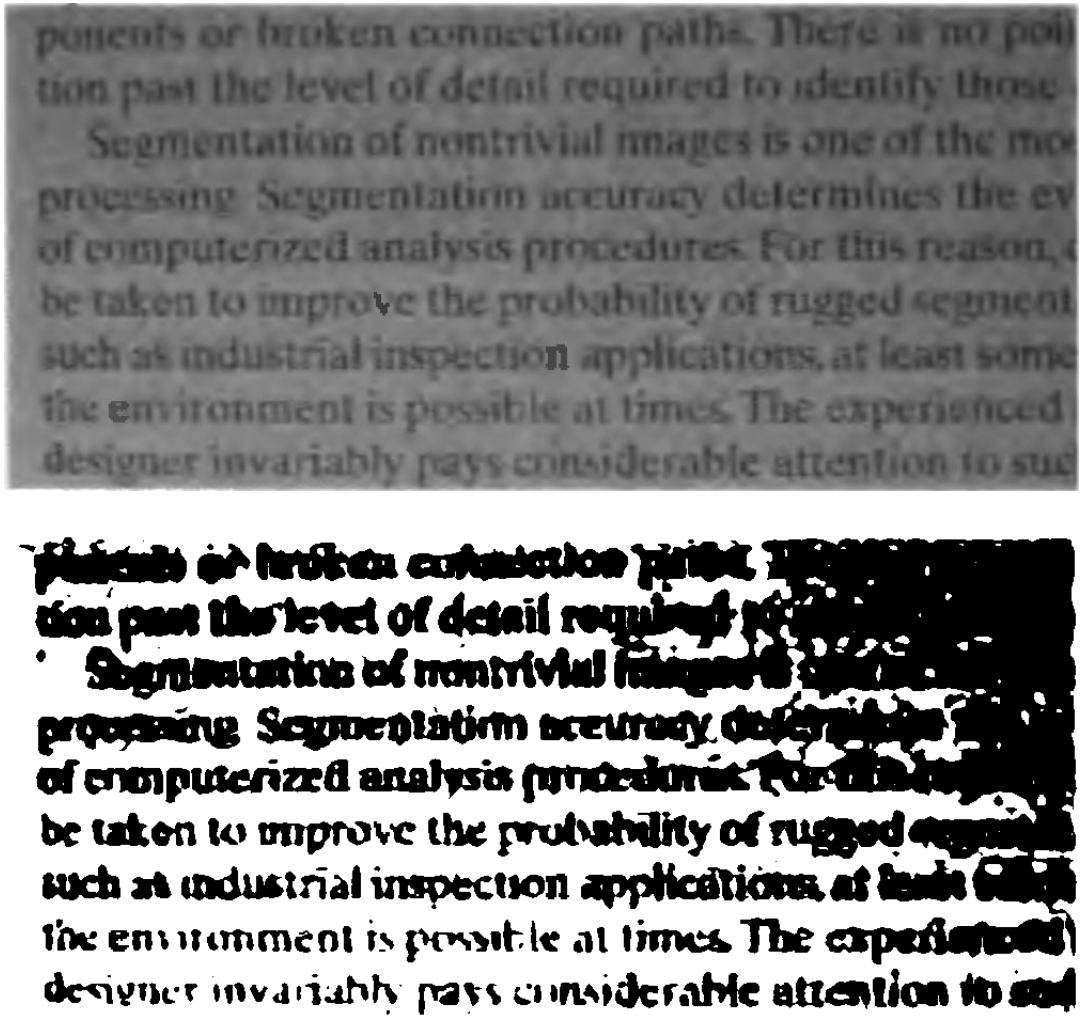




Глобальная пороговая обработка. Метод Отсу

Бинаризация изображения с помощью глобального порога не всегда даёт удовлетворительные результаты.

t=graythresh(F) =105
im2bw(F,t)
(обработка с помощью глобального порога, вычисленного по методу Отсу в системе Matlab)



Локальная пороговая обработка

Локальная пороговая обработка характеризуется тем, что изображение разбивается на подобласти, в каждой из которых для сегментации используется свое значение порога.
Основные проблемы:
- как разбить исходное изображение;
- как оценить порог для каждой полученной области.

Методы:
- Метод Бернсена.
- Метод Чоу и Канеко.
- Метод Эйквила.
- Метод Ниблацка.

Локальная пороговая обработка. Метод Бернсена.

Все изображение делится на квадраты r x r (r – нечетное) с центром в точке (m, n). Для каждого пикселя изображения в пределах квадрата используется порог, вычисляемый по формуле:

$$t(m, n) = \frac{j_{low} + j_{high}}{2}$$

где j_{low} и j_{high} являются соответственно наименьшим и наибольшим уровнем яркости в квадрате.
Если в принятой области выполняется условие

$$C(m, n) = (j_{high} - j_{low}) \leq \varepsilon$$

где ε - заданная пороговая величина, то тогда исследуемый квадрат содержит объекты только одного класса: объекты или фон.

Исследования показывают, что наилучшие результаты достигаются для значений $\varepsilon = 15$ и $r = 15$.

Локальная пороговая обработка. Метод Ниблацка.

Для каждого пикселя изображения используется свое значение порога. Определение величины порога происходит на основе вычисления локального среднего и локального среднеквадратического отклонения. Значение порога в точке с координатами (m, n) вычисляется в соответствии с формулой:

$$t(m, n) = \mu(m, n) + k\,\sigma(m, n)$$

где $\mu(m, n)$ – среднее, а $\sigma(m, n)$ – среднеквадратическое отклонение в локальной окрестности точки изображения (m, n).

Размер окрестности должен выбираться таким образом, чтобы, с одной стороны, при обработке сохранялись локальные детали, а с другой – чтобы можно было устранить шумы. Экспериментальные исследования показывают, что для окна $r = 15 \times 15$ и коэффициента $k = -0.2$ удается получить на изображении хорошо разделенные объекты и фон.

Адаптивная пороговая обработка

Для зашумленных изображений, имеющих низкое качество, например, изображений, имеющих неоднородные или изменяющиеся яркости фона, или же изображений, характеризующихся сложной текстурой фона, большой проблемой является то обстоятельство, что ни один из алгоритмов пороговой обработки не может дать удовлетворительных результатов в случае присутствия на изображении различных видов помех.

Адаптивная пороговая обработка

$$f'(m,n) = \begin{cases} 1, & \left|\hat{P}_l - f(m,n)\right| > t, \quad l = 0,\ldots,7; \\ 0, & \textit{иначе} \end{cases}$$

$$\begin{bmatrix} P_1(m_1,n_1) & P_2(m_2,n_2) & P_3(m_3,n_3) \\ P_0(m_0,n_0) & f_{mn} & P_4(m_4,n_4) \\ P_7(m_7,n_7) & P_6(m_6,n_6) & P_5(m_5,n_5) \end{bmatrix}$$

$$\hat{P}_l = \frac{1}{(2K+1)^2} \sum_{-K \le i,j \le K} f(m_l + i,\ n_l + j)$$

$\textit{средняя локальная яркость}$

$2K+1 - \textit{размер окна}$

Адаптивная пороговая обработка

Вычисление порога *t*:

1. Вычисляем

$$f_{\max} = \max_{-K \le i,j \le K} f(m+i,\ n+j) \qquad f_{\min} = \min_{-K \le i,j \le K} f(m+i,\ n+j)$$

$$\hat{P} = \frac{1}{(2K+1)^2} \sum_{-K \le i,j \le K} f(m+i,\ n+j)$$

$$\Delta f_{\max} = \left| f_{\max} - \hat{P} \right| \qquad \Delta f_{\min} = \left| f_{\min} - \hat{P} \right|$$

Адаптивная пороговая обработка

Вычисление порога t:

2. Если $\Delta f_{\max} > \Delta f_{\min}$

то $t = \alpha\left(\frac{2}{3} f_{\min} + \frac{1}{3}\hat{P}\right)$

3. Если $\Delta f_{\max} < \Delta f_{\min}$

то $t = \alpha\left(\frac{1}{3} f_{\min} + \frac{2}{3}\hat{P}\right)$

Адаптивная пороговая обработка

Вычисление порога *t*:

4. Если $\Delta f_{\max} = \Delta f_{\min}$ и $f_{\max} \neq f_{\min}$
То увеличиваем размер окна на 1 (K=K+1) и повторяем шаги 1-3.
Если при этом $f_{\max} = f_{\min}$, то $t = \alpha \hat{P}$

α - константа из диапазона [0.3, …, 0.8].
Для изображений очень плохого качества с большим шумом и низким контрастом следует использовать значение $\alpha = 1/3$. В большинстве же случаев достаточным является значение $\alpha = 2/3$

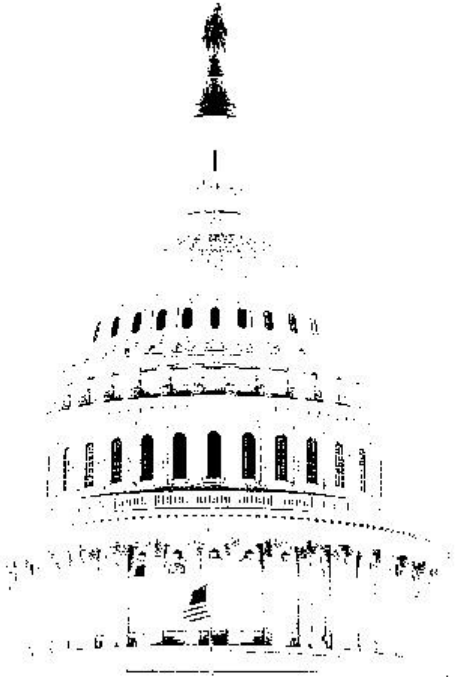
Адаптивная пороговая обработка

Пример: бинаризация с нижним порогом.

исходное



t=128



t=190



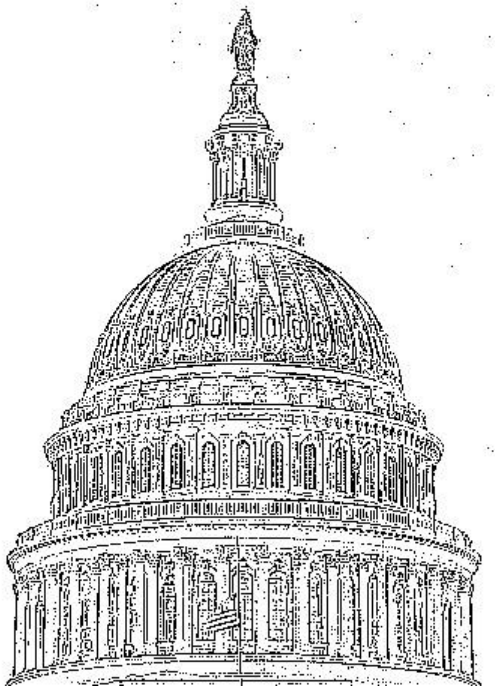
Адаптивная пороговая обработка

Пример: адаптивная пороговая обработка.

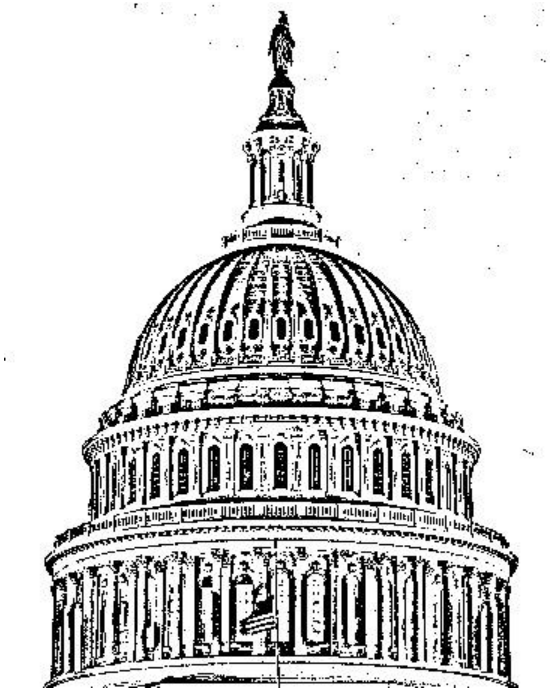
Исходное



CV_ADAPTIVE_THRESH_MEAN_C
block_size=3



block_size=11



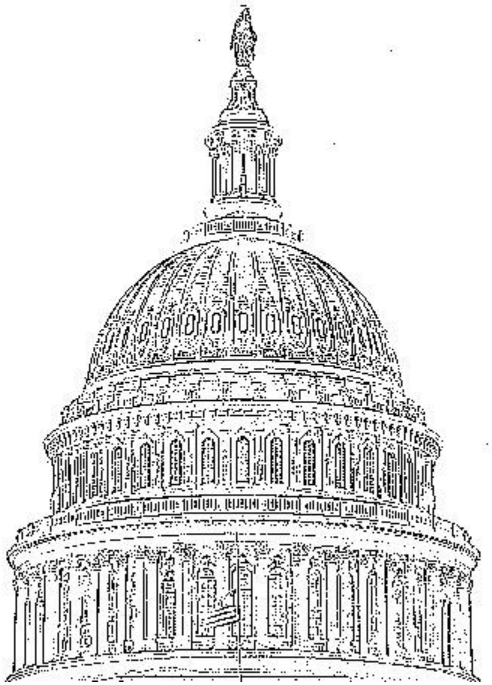
Адаптивная пороговая обработка

Пример: адаптивная пороговая обработка.

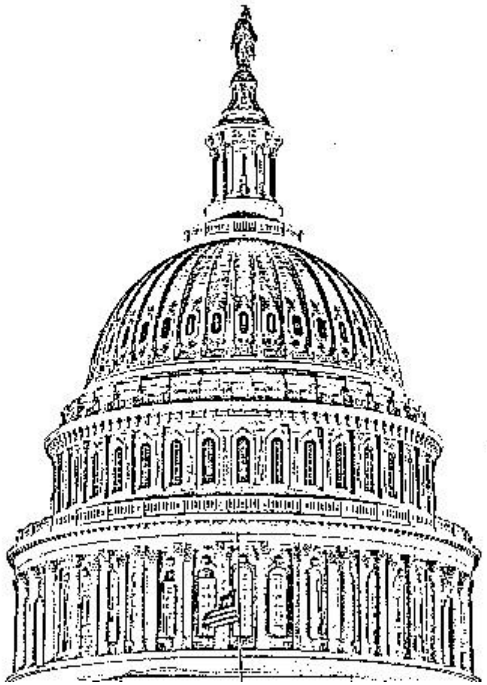
Исходное



CV_ADAPTIVE_THRESH_GAUSSIAN_C
block_size=3



block_size=11



Адаптивная пороговая обработка





OpenCV

OpenCV (англ. *Open Source Computer Vision Library*, библиотека компьютерного зрения с открытым исходным кодом) — библиотека алгоритмов компьютерного зрения, обработки изображений и численных алгоритмов общего назначения с открытым кодом.

Реализована на C/C++, официальные биндинги для Python, Java, JavaScript (OpenCV.js). Актуальная версия: 4.x (4.10.0, 2024).



OpenCV

- ***opencv_core*** — основная функциональность. Включает в себя базовые структуры, вычисления(математические функции, генераторы случайных чисел) и линейную алгебру, и т. д.
- ***opencv_imgproc*** — обработка изображений (фильтрация, геометрические преобразования, преобразование цветовых пространств и т. д.).
- ***opencv_highgui*** — простой UI, ввод/вывод изображений и видео.
- ***opencv_ml*** — модели машинного обучения (SVM, деревья решений, обучение со стимулированием и т. д.).



OpenCV

- ***opencv_features2d*** — распознавание и описание плоских примитивов.
- ***opencv_video*** — анализ движения и отслеживание объектов (оптический поток, шаблоны движения, устранение фона).
- ***opencv_objdetect*** — обнаружение объектов на изображении (нахождение лиц с помощью алгоритма Виолы-Джонса, распознавание людей HOG и т. д.).
- ***opencv_calib3d*** — калибровка камеры, поиск стерео-соответствия и элементы обработки трехмерных данных.
- ***opencv_dnn —*** загрузка и запуск нейронных сетей (TensorFlow, PyTorch, ONNX); основной модуль для современных задач обнаружения объектов и сегментации.



OpenCV

Где используется:

- Google Maps, Google street view, Google Earth.
- Научные и промышленные исследования.
- Медицинская визуализация.
- Системы обеспечения безопасности.
- Поиск изображений.
- Видео-поиск.
- Мониторинг конвейерной продукции в производственных системах.
- Робототехника.